SaiGon University - February 2026

PHÂN TÍCH VÀ PHÂN LOẠI CÁC GIỐNG HOA IRIS

Exploratory Data Analysis

Machine Learning

Botany

TỔNG QUAN VỀ BÀI TOÁN

Hoa Iris (Diên vĩ) là gì?

- Loài thực vật phổ biến trong nghiên cứu sinh học.
- Đề tài tập trung vào 3 giống chính: Setosa, Versicolor và Virginica.

Tại sao cần phân loại tự động?

- Phân loại thủ công bằng mắt thường tốn thời gian và dễ sai sót.
- Ứng dụng Học máy (Machine Learning) giúp tự động hóa nhận diện, hỗ trợ hệ thống nông nghiệp thông minh.

Bộ dữ liệu Iris (The Iris Dataset)

- Tập dữ liệu "kinh điển" trong giới Trí tuệ nhân tạo.
- Quy mô: 150 mẫu chuẩn hóa.
- Đặc trưng đầu vào: Kích thước chiều dài/rộng của đài hoa (Sepal) và cánh hoa (Petal).

MỤC TIÊU NGHIÊN CỨU

MỤC TIÊU CHÍNH

💡 Thực hiện quá trình rút trích đặc trưng và tiền xử lý toàn diện cho bộ dữ liệu hoa Iris

MỤC TIÊU NGHIÊN CỨU

MỤC TIÊU CỤ THỂ

- **Khám phá dữ liệu (EDA)**: Phân tích, trực quan hóa và đánh giá sự phân bố của các đặc trưng (Sepal, Petal).
- **Làm sạch dữ liệu (Data Cleaning):** Kiểm tra tính toàn vẹn, xử lý giá trị khuyết thiếu (Missing values) và loại bỏ dữ liệu trùng lặp.
- **Biến đổi dữ liệu (Data Transformation):** Thực hiện mã hóa nhãn (Label Encoding) cho biến mục tiêu.
- **Đóng gói kết quả:** Xuất ra bộ dữ liệu "sạch" và các file chuẩn hóa (joblib, xlsx)

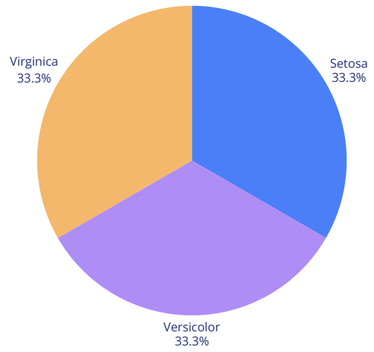
TỔNG QUAN DỮ LIỆU (THE IRIS DATASET)

Nguồn gốc:

- Thu thập thực tế (1936): Được đo lường và ghi chép thủ công bởi nhà thực vật học Edgar Anderson từ các mẫu hoa thật.
- Công bố khoa học: Nổi tiếng qua bài báo nền tảng về "Phân loại học" của nhà thống kê học Ronald Fisher.

Phân bố biến mục tiêu:

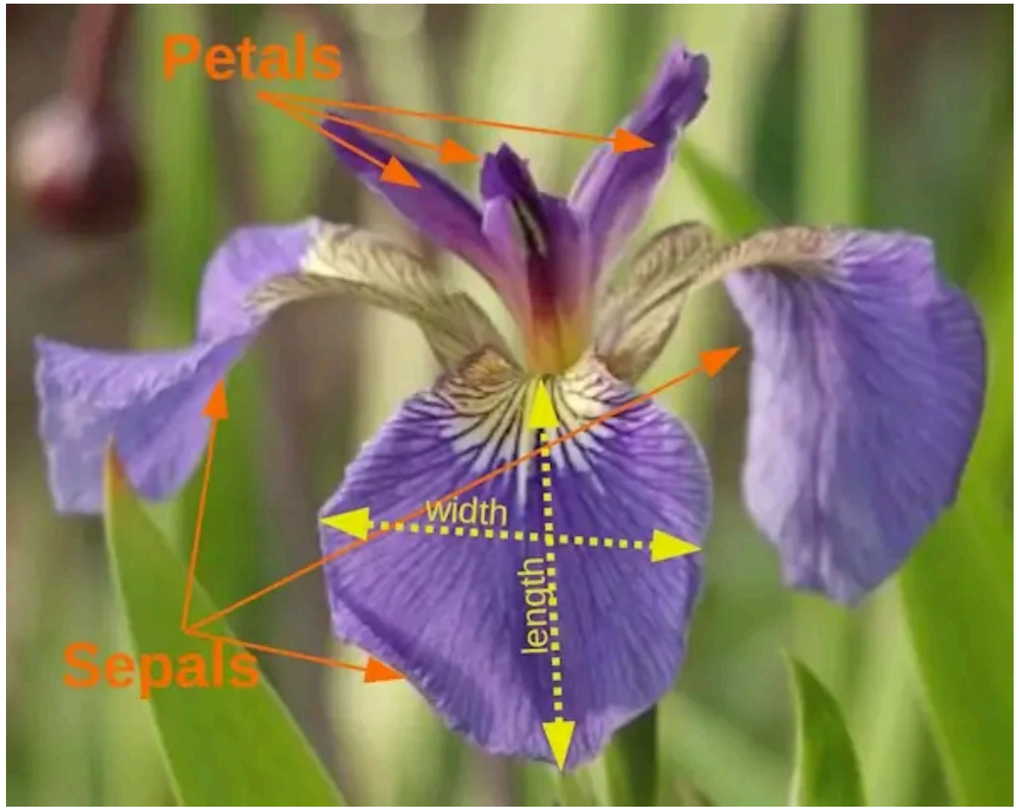
- Biến mục tiêu: class (0, 1, 2)
- 0: Setosa (50 hoa, 33,3%)
- 1: Versicolor (50 hoa, 33,3%)
- 2: Virginica (50 hoa, 33,3%)



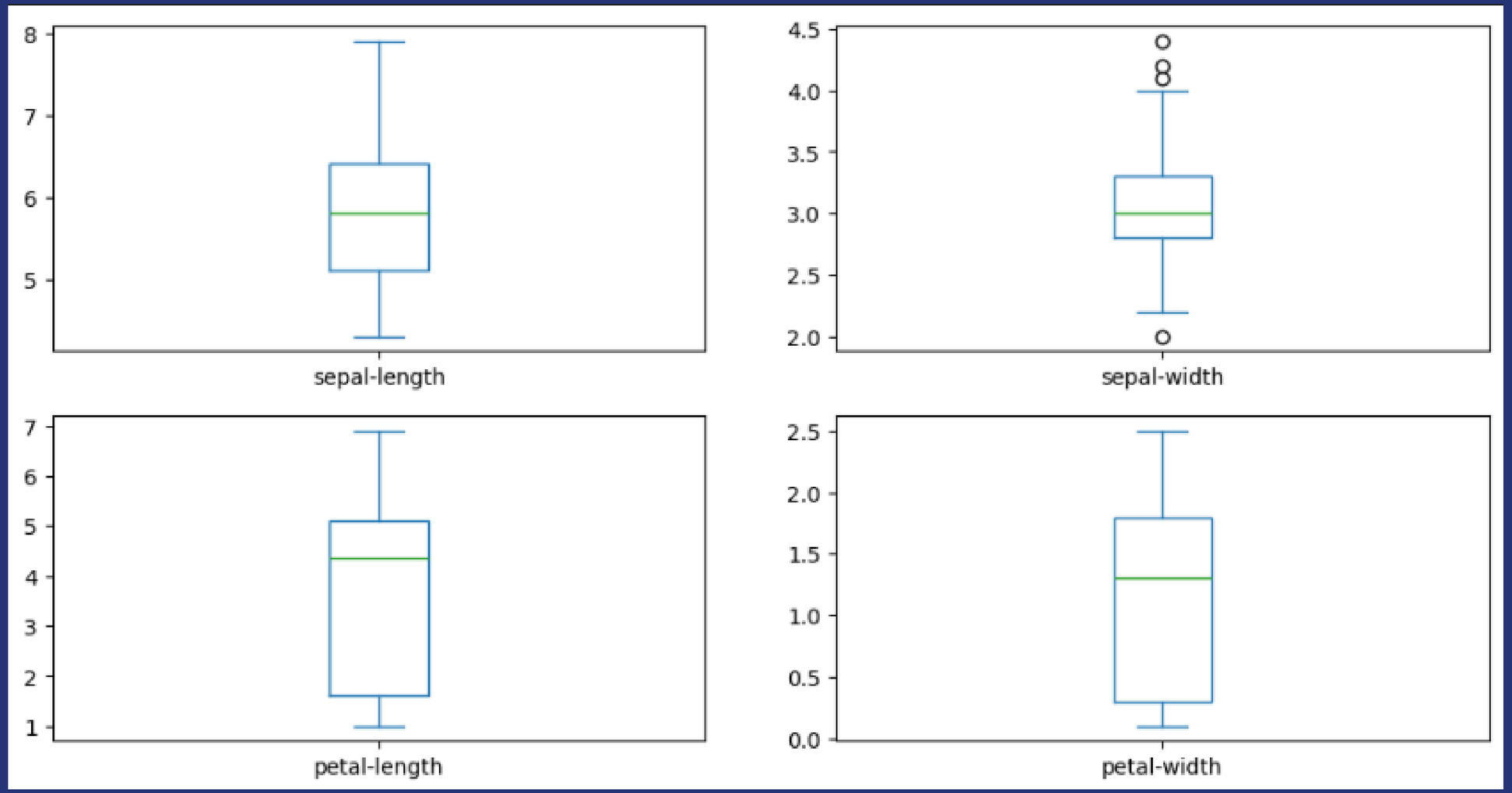
TỔNG QUAN DỮ LIỆU (THE IRIS DATASET)

TỪ ĐIỂN DỮ LIỆU:

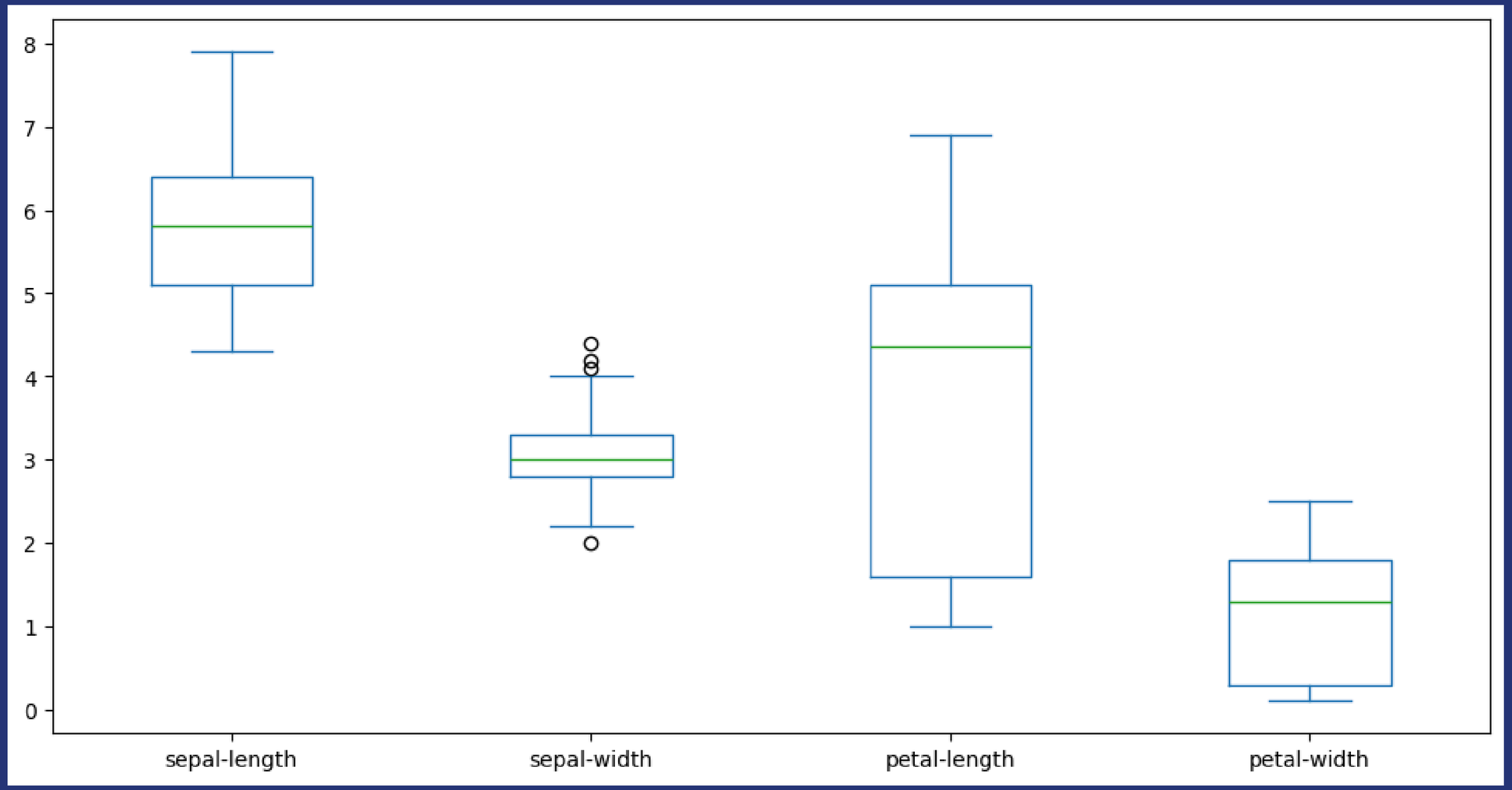
- Đặc trưng đầu vào (Features - X):
  - Đài hoa (Sepal):
    - sepal-length (Chiều dài) & sepal-width (Chiều rộng).
  - Cánh hoa (Petal):
    - petal-length (Chiều dài) & petal-width (Chiều rộng).
  - Đơn vị đo lường chung: Centimet (cm).
- Đầu ra (Target - Y): class (Giống hoa đã mã hóa 0, 1, 2).



BIỂU ĐỒ HỘP (BOXPLOT)



BIỂU ĐỒ HỘP (BOXPLOT)



BIỂU ĐỒ HỘP (BOXPLOT)

📏 **Sự phân tán dữ liệu (Data Spread):**

- Độ trải rộng (biên độ dao động) giữa 4 thuộc tính có sự chênh lệch và khác biệt rõ rệt.

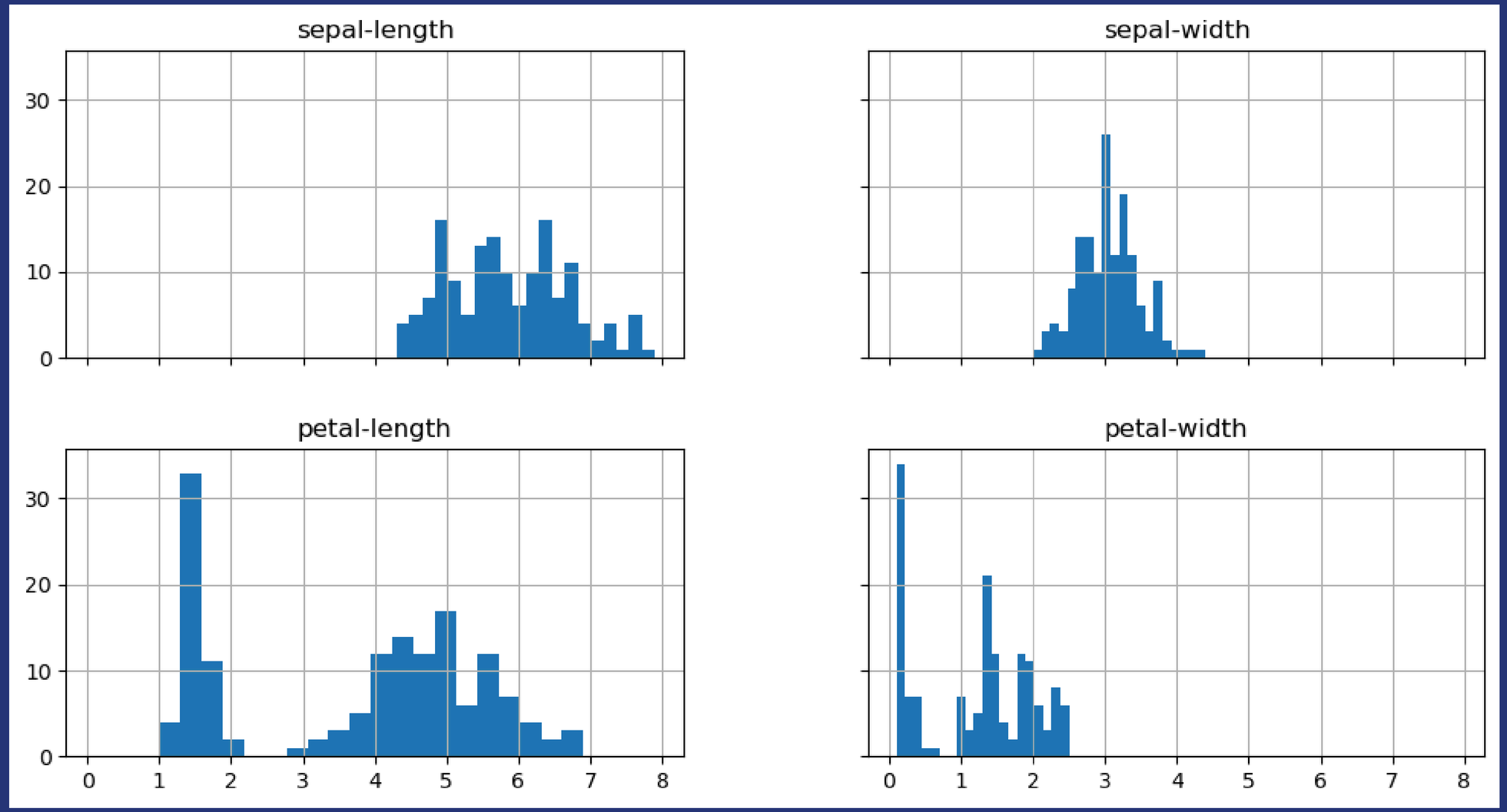
📉 **Độ lệch phân bố (Skewness):**

- Các thuộc tính sepal-length, petal-length và petal-width có xu hướng phân bố lệch về hướng các giá trị nhỏ. (Nghĩa là phần lớn các mẫu hoa đều tập trung ở kích thước bé).

⚖️ **Tính cân bằng (Symmetry):**

- Riêng đặc trưng sepal-width (Chiều rộng đài hoa) có phân bố giá trị rất cân bằng và đối xứng (đường trung vị nằm gần như chính giữa dải dữ liệu).

BIỂU ĐỒ HISTOGRAM



KIỂM TRA TÍNH TOÀN VẸN & LÀM SẠCH DỮ LIỆU

Kiểm tra giá trị khuyết thiếu (Missing Values):

- Sử dụng hàm kiểm tra toàn bộ ma trận dữ liệu.
- Kết quả: Không phát hiện bất kỳ giá trị Null hay NaN nào. Dữ liệu gốc khá hoàn thiện.

Xử lý dữ liệu trùng lặp (Duplicates):

- Quét các mẫu dữ liệu có thông số giống nhau y hệt.
- Giải pháp: Sử dụng lệnh loại bỏ trùng lặp để giữ lại tập dữ liệu tinh gọn và độc nhất.

KIỂM TRA TÍNH TOÀN VẸN & LÀM SẠCH DỮ LIỆU

(1) Tạo bảng dữ liệu làm sạch

- Chỉ giữ lại các cột Input, Ouput

```
df_clean = df_dataset.copy()
```

✓ 0.0s

KIỂM TRA TÍNH TOÀN VẸN & LÀM SẠCH DỮ LIỆU

2) Xóa dữ liệu trùng nhau

```
display.display(df_dataset[df_dataset.duplicated()])
```

	sepal-length	sepal-width	petal-length	petal-width	class
34	4.9	3.1	1.5	0.1	Iris-setosa
37	4.9	3.1	1.5	0.1	Iris-setosa
142	5.8	2.7	5.1	1.9	Iris-virginica

```
s_query1 = "`sepal-length`==4.9 and `sepal-width`==3.1 and `class`=='Iris-setosa'"
s_query2 = "`sepal-length`==5.8 and `sepal-width`==2.7 and `class`=='Iris-virginica'
display.display(df_dataset.query(s_query1))
display.display(df_dataset.query(s_query2))
```

	sepal-length	sepal-width	petal-length	petal-width	class
9	4.9	3.1	1.5	0.1	Iris-setosa
34	4.9	3.1	1.5	0.1	Iris-setosa
37	4.9	3.1	1.5	0.1	Iris-setosa

	sepal-length	sepal-width	petal-length	petal-width	class
101	5.8	2.7	5.1	1.9	Iris-virginica
142	5.8	2.7	5.1	1.9	Iris-virginica

KIỂM TRA TÍNH TOÀN VẸN & LÀM SẠCH DỮ LIỆU

```python
df_clean.drop_duplicates(ignore_index=True, inplace=True)
df_clean
```

	sepal-length	sepal-width	petal-length	petal-width	class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...
142	6.7	3.0	5.2	2.3	Iris-virginica
143	6.3	2.5	5.0	1.9	Iris-virginica
144	6.5	3.0	5.2	2.0	Iris-virginica
145	6.2	3.4	5.4	2.3	Iris-virginica
146	5.9	3.0	5.1	1.8	Iris-virginica

147 rows × 5 columns

(3) Xử lý giá trị rỗng, không hợp lệ

```python
has_null = df_dataset.isnull().sum().any()
has_nan  = df_dataset.isna().sum().any()
print(f'+ Có giá trị Null: {has_null}')
if has_null:
    display.display(df_dataset[df_dataset.isnull().any(axis=1)])
print(f'+ Có giá trị Nan: {has_nan}')
if has_nan:
    display.display(df_dataset[df_dataset.isna().any(axis=1)])
```

```
+ Có giá trị Null: False
+ Có giá trị Nan: False
```

BIẾN ĐỔI DỮ LIỆU

Chuyển đổi dữ liệu danh mục (Category) thành dữ liệu số

Chuyển đổi dữ liệu danh mục (Category) thành dạng OneHot

Chuẩn hóa dữ liệu (Data Normalize)

Chia dữ liệu thực nghiệm

CHUYỂN ĐỔI DỮ LIỆU DANH MỤC (CATEGORY) THÀNH DỮ LIỆU SỐ

⚠️ Vấn đề: Các thuật toán Học máy chỉ xử lý được ma trận số học, không đọc được dữ liệu dạng chuỗi (Text).

- Giải pháp: Thực hiện mã hóa biến mục tiêu class.
- Kết quả chuyển đổi:
  - Iris-setosa => 0
  - Iris-versicolor => 1
  - Iris-virginica => 2

CHUYỂN ĐỔI DỮ LIỆU DANH MỤC (CATEGORY) THÀNH DỮ LIỆU SỐ

4.2. Biến đổi dữ liệu (Data Transforms)

(1) Chuyển đổi dữ liệu danh mục (Category) thành dữ liệu số

```python
class_encoder = LabelEncoder()
class_encoder.fit(df_clean["class"])
class_names = class_encoder.classes_
print(class_names)
```

Python

```
['Iris-setosa' 'Iris-versicolor' 'Iris-virginica']
```

```python
class_encoder.transform(df_clean["class"])
```

Python

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

CHUYỂN ĐỔI DỮ LIỆU DANH MỤC (CATEGORY) THÀNH DỮ LIỆU SỐ

```python
df_clean["class"] = class_encoder.transform(df_clean["class"])
df_clean
```

Python

	sepal-length	sepal-width	petal-length	petal-width	class
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...
142	6.7	3.0	5.2	2.3	2
143	6.3	2.5	5.0	1.9	2
144	6.5	3.0	5.2	2.0	2
145	6.2	3.4	5.4	2.3	2
146	5.9	3.0	5.1	1.8	2

147 rows × 5 columns

CHUYỂN ĐỔI DỮ LIỆU DANH MỤC (CATEGORY) THÀNH DẠNG ONEHOT

- Một số thuật toán khi chuyển đổi cột dạng danh mục thành kiểu OneHot thì cho hiệu suất cao hơn.
- Bên cạnh đó, khi huấn luyện mô hình với dạng hàm mất mát CategoryEntropy thì cũng cần chuyển thuộc tính phân lớp sang dạng OneHot.

CHUYỂN ĐỔI DỮ LIỆU DANH MỤC (CATEGORY) THÀNH DẠNG ONEHOT

- Một số thuật toán khi chuyển đổi cột dạng danh mục thành kiểu OneHot thì cho hiệu suất cao hơn.
- Bên cạnh đó, khi huấn luyện mô hình với dạng hàm mất mát CategoryEntropy thì cũng cần chuyển thuộc tính phân lớp sang dạng OneHot.

CHUYỂN ĐỔI DỮ LIỆU DANH MỤC (CATEGORY) THÀNH DẠNG ONEHOT

Chuẩn hóa các tính chất để đưa về cùng một miền trị

- Min-Max Normalization

$$z = \frac{x - min(x)}{max(x) - min(x)}$$

- Standard Normalization

$$z = \frac{x - \mu}{\sigma}$$

Lưu ý: Quá trình chuẩn hóa có thể làm trong phần thực nghiệm thuật toán

CHUYỂN ĐỔI DỮ LIỆU DANH MỤC (CATEGORY) THÀNH DẠNG ONEHOT

```python
# Min-Max Normalization
scale_columns = ["sepal-length", "sepal-width", "petal-length", "petal-width"]
minmax_scaler = MinMaxScaler()
minmax_scaler.fit(df_clean[scale_columns])
df_clean_minmax = df_clean.copy()
df_clean_minmax[scale_columns] = minmax_scaler.transform(df_clean[scale_columns])
df_clean_minmax
```

✓ 0.0s

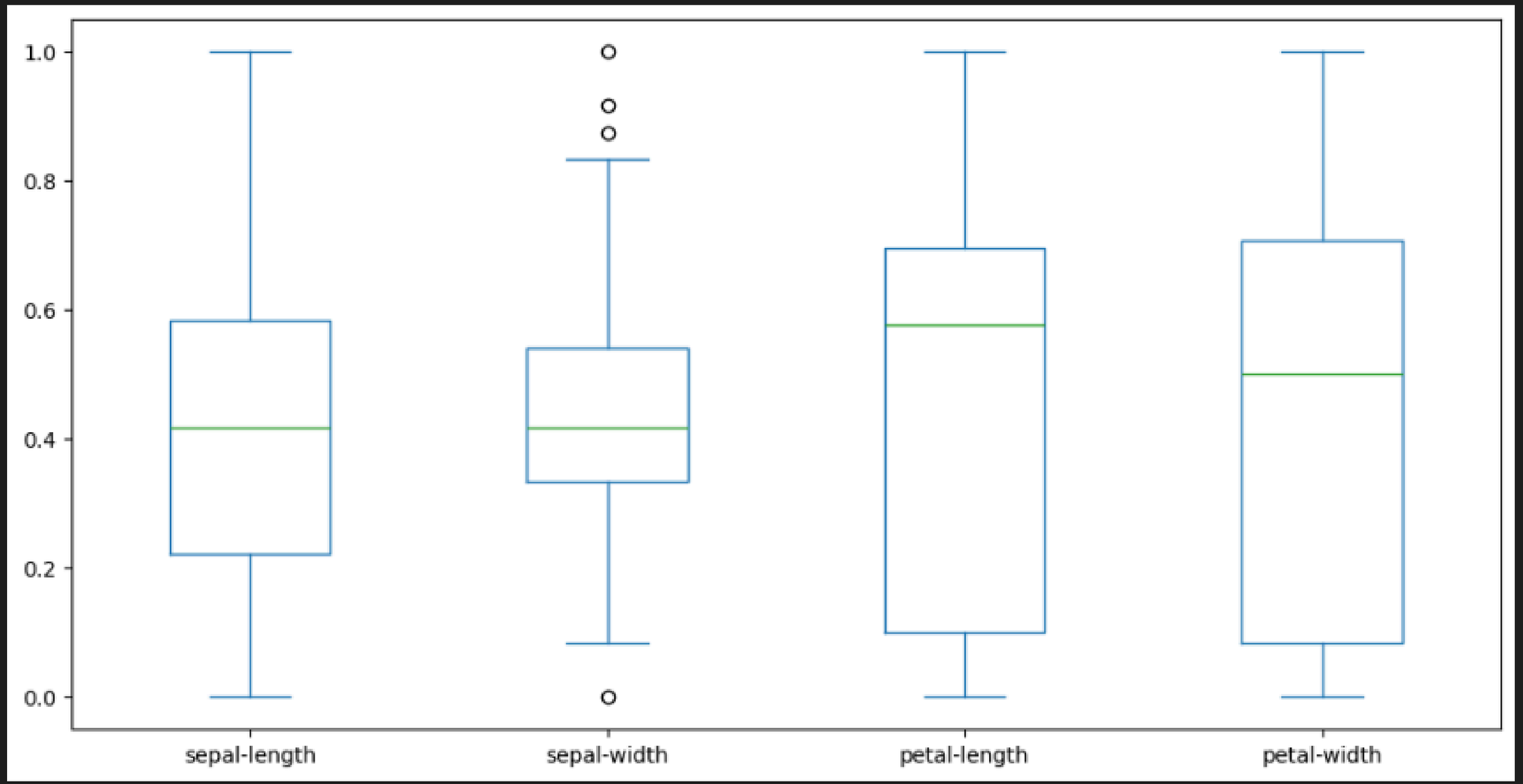
	sepal-length	sepal-width	petal-length	petal-width	class
0	0.222222	0.625000	0.067797	0.041667	0
1	0.166667	0.416667	0.067797	0.041667	0
2	0.111111	0.500000	0.050847	0.041667	0
3	0.083333	0.458333	0.084746	0.041667	0
4	0.194444	0.666667	0.067797	0.041667	0
...	...	...	...	...	...
142	0.666667	0.416667	0.711864	0.916667	2
143	0.555556	0.208333	0.677966	0.750000	2
144	0.611111	0.416667	0.711864	0.791667	2
145	0.527778	0.583333	0.745763	0.916667	2
146	0.444444	0.416667	0.694915	0.708333	2

147 rows × 5 columns

CHUYỂN ĐỔI DỮ LIỆU DANH MỤC (CATEGORY) THÀNH DẠNG ONEHOT

```
df_clean_minmax[scale_columns].plot(kind="box", figsize=(12,6));
```

✓ 0.0s



CHUYỂN ĐỔI DỮ LIỆU DANH MỤC (CATEGORY) THÀNH DẠNG ONEHOT

```python
# Standard Normalization
standard_scaler = StandardScaler()
standard_scaler.fit(df_clean[scale_columns])
df_clean_standard = df_clean.copy()
df_clean_standard[scale_columns] = standard_scaler.transform(df_clean[scale_columns])
df_clean_standard
```

✓ 0.0s

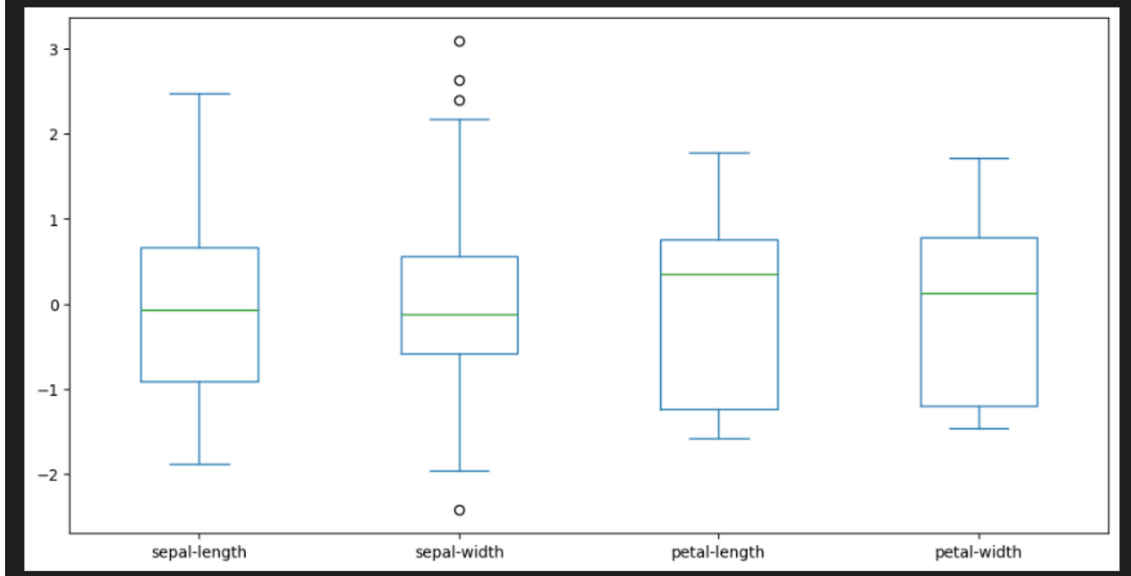
	sepal-length	sepal-width	petal-length	petal-width	class
0	-0.915509	1.019971	-1.357737	-1.335700	0
1	-1.157560	-0.128082	-1.357737	-1.335700	0
2	-1.399610	0.331139	-1.414778	-1.335700	0
3	-1.520635	0.101529	-1.300696	-1.335700	0
4	-1.036535	1.249582	-1.357737	-1.335700	0
...	...	...	...	...	...
142	1.020892	-0.128082	0.809831	1.444682	2
143	0.536792	-1.276136	0.695748	0.915085	2
144	0.778842	-0.128082	0.809831	1.047484	2
145	0.415766	0.790361	0.923913	1.444682	2
146	0.052691	-0.128082	0.752789	0.782686	2

147 rows × 5 columns

CHUYỂN ĐỔI DỮ LIỆU DANH MỤC (CATEGORY) THÀNH DẠNG ONEHOT

```
df_clean_standard[scale_columns].plot(kind="box", figsize=(12,6));
```

✓ 0.0s




```
correlations = df_clean_standard.corr(method='pearson')
display.display(correlations)
```

✓ 0.0s

	sepal-length	sepal-width	petal-length	petal-width	class
sepal-length	1.000000	-0.109321	0.871305	0.817058	0.782904
sepal-width	-0.109321	1.000000	-0.421057	-0.356376	-0.418348
petal-length	0.871305	-0.421057	1.000000	0.961883	0.948339
petal-width	0.817058	-0.356376	0.961883	1.000000	0.955693
class	0.782904	-0.418348	0.948339	0.955693	1.000000

CHIA DỮ LIỆU THỰC NGHIỆM

- Chuyển đổi dữ liệu sang dạng numpy với phần Input (X_data), Output (y_data)
- Chia dữ liệu thành tập train/test (tỷ lệ 70/30)
- Lưu tất cả thông tin để chuẩn bị chạy thuật toán

CHIA DỮ LIỆU THỰC NGHIỆM

```
Click to add a breakpoint a = df_clean.values[:, :-1], df_clean.values[:, -1]
X_data.shape, y_data.shape
print(X_data[:20, :])
print(y_data[:20])
```

✓ 0.0s

```
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]
 [5.4 3.9 1.7 0.4]
 [4.6 3.4 1.4 0.3]
 [5.  3.4 1.5 0.2]
 [4.4 2.9 1.4 0.2]
 [4.9 3.1 1.5 0.1]
 [5.4 3.7 1.5 0.2]
 [4.8 3.4 1.6 0.2]
 [4.8 3.  1.4 0.1]
 [4.3 3.  1.1 0.1]
 [5.8 4.  1.2 0.2]
 [5.7 4.4 1.5 0.4]
 [5.4 3.9 1.3 0.4]
 [5.1 3.5 1.4 0.3]
 [5.7 3.8 1.7 0.3]
 [5.1 3.8 1.5 0.3]]
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

CHIA DỮ LIỆU THỰC NGHIỆM

```python
X_train, X_test, y_train, y_test = train_test_split(X_data, y_data, train_size=0.7)
print(f'+ Train/Test: Train Ratio = {len(X_train)/len(X_data)}')
print(f'+ Train: shape={X_train.shape}\nInput = \n{X_train[:5]}\nOutput = {y_train[:5]}')
print(f'+ Test: shape={X_test.shape}\nInput = \n{X_test[:5]}\nOutput = {y_test[:5]}')
```

✓ 0.0s

```
+ Train/Test: Train Ratio = 0.6938775510204082
+ Train: shape=(102, 4)
Input =
[[5.7 2.6 3.5 1. ]
 [6.7 3.  5.  1.7]
 [4.9 3.  1.4 0.2]
 [6.7 3.1 4.7 1.5]
 [6.8 2.8 4.8 1.4]]
Output = [1. 1. 0. 1. 1.]
+ Test: shape=(45, 4)
Input =
[[6.3 2.5 4.9 1.5]
 [5.9 3.  5.1 1.8]
 [5.  3.4 1.5 0.2]
 [6.3 2.7 4.9 1.8]
 [6.4 3.2 5.3 2.3]]
Output = [1. 2. 0. 2. 2.]
```

CHIA DỮ LIỆU THỰC NGHIỆM

```
pd.read_excel(f'{save_dir}/df_clean.xlsx')
```

✓ 0.0s

	sepal-length	sepal-width	petal-length	petal-width	class
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...
142	6.7	3.0	5.2	2.3	2
143	6.3	2.5	5.0	1.9	2
144	6.5	3.0	5.2	2.0	2
145	6.2	3.4	5.4	2.3	2
146	5.9	3.0	5.1	1.8	2

147 rows × 5 columns

```
joblib.load(f'{save_dir}/class_encoder.joblib').classes_
```

✓ 0.0s

```
array(['Iris-setosa', 'Iris-versicolor', 'Iris-virginica'], dtype=object)
```

CHIA DỮ LIỆU THỰC NGHIỆM

```python
X_train, X_test, y_train, y_test = train_test_split(X_data, y_data, train_size=0.7)
print(f'+ Train/Test: Train Ratio = {len(X_train)/len(X_data)}')
print(f'+ Train: shape={X_train.shape}\nInput = \n{X_train[:5]}\nOutput = {y_train[:5]}')
print(f'+ Test: shape={X_test.shape}\nInput = \n{X_test[:5]}\nOutput = {y_test[:5]}')
```

✓ 0.0s

```
+ Train/Test: Train Ratio = 0.6938775510204082
+ Train: shape=(102, 4)
Input =
[[5.7 2.6 3.5 1. ]
 [6.7 3.  5.  1.7]
 [4.9 3.  1.4 0.2]
 [6.7 3.1 4.7 1.5]
 [6.8 2.8 4.8 1.4]]
Output = [1. 1. 0. 1. 1.]
+ Test: shape=(45, 4)
Input =
[[6.3 2.5 4.9 1.5]
 [5.9 3.  5.1 1.8]
 [5.  3.4 1.5 0.2]
 [6.3 2.7 4.9 1.8]
 [6.4 3.2 5.3 2.3]]
Output = [1. 2. 0. 2. 2.]
```

THANK YOU